

Детальний аналіз функцій проєкту

Лабораторна робота №1: Комп'ютерна графіка та візуалізація

1. Модуль `manual\_algorithms.py` (Власні математичні алгоритми)

У цьому модулі зібрано «чисті» алгоритми обробки масивів пікселів без використання сторонніх бібліотек.

Функція: `resize_nearest_neighbor(pixels, orig_w, orig_h, new_w, new_h)`

- **Призначення:** Зміна роздільної здатності зображення методом найближчого сусіда (Nearest Neighbor).

- **Параметри:**

- `pixels`: одновимірний список кортежів `[(R, G, B), ...]`.
- `orig_w, orig_h`: початкова ширина та висота (в пікселях).
- `new_w, new_h`: цільова ширина та висота.

- **Повертає:** новий список пікселів `[(R, G, B), ...]` розміром `new_w * new_h`.

- **Принцип роботи:**

1. Обчислюються масштабні коефіцієнти: `x_ratio = orig_w / new_w`, `y_ratio = orig_h / new_h`.
2. Для кожної цільової координати `(x, y)` шукається найближчий цілий індекс у вихідному масиві: `orig_x = min(orig_w - 1, int(x * x_ratio))`, `orig_y = min(orig_h - 1, int(y * y_ratio))`.
3. Значення кольору просто копіюється з комірки `orig_y * orig_w + orig_x`.

- **Як пояснити викладачу:**

Пояснення для захисту: «Це найпростіший і найшвидший метод масштабування. Він не виконує жодного усереднення, а бере колір пікселя, який знаходиться найближче до обчисленої дробової координати. При збільшенні дає характерну пікселізацію («сходинки»), але ідеально зберігає чіткість контрастних ліній.»

Функція: `resize_bilinear(pixels, orig_w, orig_h, new_w, new_h)`

- **Призначення:** Зміна розміру зображення методом білінійної інтерполяції (плавне масштабування).

- **Параметри:**

- `pixels`: вхідний список пікселів `[(R, G, B), ...]`.
- `orig_w, orig_h`: початкові розміри.
- `new_w, new_h`: нові розміри.

- **Повертає:** список згладжених пікселів нового розміру.

- **Принцип роботи:**

1. Розраховуються дробові координати у вихідному зображенні `src_x, src_y`.
2. Знаходяться 4 сусідні опорні точки: `(x1, y1)`, `(x2, y1)`, `(x1, y2)`, `(x2, y2)` та їхні кольори `P_{11}`, `P_{21}`, `P_{12}`, `P_{22}`.
3. Обчислюються відстані-ваги $dx = \text{src_x} - x_1$, $dy = \text{src_y} - y_1$.
4. Для кожного колірної каналу (R, G, B) рахується зважена сума за формулою:
$$C = (1-dx)(1-dy) P_{11} + dx(1-dy) P_{21} + (1-dx)dy P_{12} + dx dy P_{22}$$
5. Отримане значення обмежується діапазоном `[0, 255]` (`clamp`).

- **Як пояснити викладачу:**

Пояснення для захисту: «Білінійна інтерполяція виконує зважене усереднення між чотирма сусідніми точками по осях X та Y. Завдяки цьому при зміні розміру градієнти та плавні переходи виглядають природно, без різких сходенок.»

Функція: `replace_pixels_color(pixels, target_rgb, new_rgb, tolerance=15)`

- **Призначення:** Попіксельний пошук і заміна кольору з урахуванням заданої похибки (`tolerance`).
- **Параметри:**
 - **`pixels`**: масив пікселів.
 - **`target_rgb`**: кортеж кольору, який шукаємо (**`R`**, **`G`**, **`B`**).
 - **`new_rgb`**: кортеж нового кольору (**`R`**, **`G`**, **`B`**).
 - **`tolerance`**: допустиме відхилення за кожним каналом (по замовчуванню 15).
- **Повертає:** (**`result_pixels`**, **`replaced_count`**) — модифікований список пікселів та кількість замінених точок.
- **Принцип роботи:**

1. Проходимо циклом по всіх пікселях масиву.
2. Перевіряємо умову: $|R - R_t| \leq \text{tolerance} \ \& \ |G - G_t| \leq \text{tolerance} \ \& \ |B - B_t| \leq \text{tolerance}$.
3. Якщо умова істинна — записуємо **`new_rgb`** та збільшуємо лічильник **`replaced_count`**. Інакше залишаємо початковий піксель (**`R`**, **`G`**, **`B`**).

- **Як пояснити викладачу:**

Пояснення для захисту: «Функція перебирає всі пікселі зображення і порівнює кожен канал з еталонним цільовим кольором. Параметр `tolerance` визначає радіус колірного вікна: якщо `tolerance = 0`, здійснюється точна заміна, якщо 0 — захоплюються близькі відтінки.»

Функція: `apply_color_balance(pixels, r_factor, g_factor, b_factor, r_delta, g_delta, b_delta)`

- **Призначення:** Ручне налаштування колірного балансу (множники та адитивні зсуви каналів).
- **Параметри:**
 - **`r_factor`**, **`g_factor`**, **`b_factor`**: коефіцієнти множення (масштабування інтенсивності, 1.0 = без змін).
 - **`r_delta`**, **`g_delta`**, **`b_delta`**: постійні зсуви (додавання/віднімання яскравості).
- **Повертає:** скоригований масив пікселів.
- **Принцип роботи:**

Для кожного каналу кожного пікселя обчислюється нова інтенсивність за формулою:

$$C_{\text{new}} = \max(0, \min(255, \text{int}(C_{\text{old}} \cdot k_C + \Delta_C)))$$

- **Як пояснити викладачу:**

Пояснення для захисту: «Ручна корекція змінює динамічний діапазон і яскравість окремих колірних каналів. Множник (`factor`) змінює контрастність та насиченість каналу, а доданок (`delta`) рівномірно зсуває яскравість у світлий або темний бік. Функція `clamp` гарантує, що значення не вийдуть за межі 8-бітного діапазону [0..255].»

Функція: `apply_auto_color_balance(pixels, target_mean=128.0)`

- **Призначення:** Автоматичне вирівнювання колірного балансу за формулою з методичних вказівок.
- **Параметри:**
 - **`pixels`**: вхідний список пікселів.
 - **`target_mean`**: цільова середня яскравість (за методичкою = 128.0).
- **Повертає:** (**`result_pixels`**, **`stats_dict`**) — оброблений масив та словник зі статистикою (початкові середні та зсуви).
- **Принцип роботи:**

1. Обчислюються середні значення інтенсивності для кожного каналу окремо:

$$\text{mean}_R = \frac{1}{N} \sum R_i, \quad \text{mean}_G = \frac{1}{N} \sum G_i, \quad \text{mean}_B = \frac{1}{N} \sum B_i$$
2. Обчислюються адитивні коригувальні зміщення: $\text{shift}_C = \text{target_mean} - \text{mean}_C$.
3. Кожен піксель зміщується на відповідний shift_C з обмеженням [0, 255].

- **Як пояснити викладачу:**

Пояснення для захисту: «Автоматичний баланс реалізовано на основі гіпотези нормалізації середнього тону: якщо середнє значення каналу менше 128, канал освітлюється, якщо більше 128 — затемнюється.

Це усуває паразитні колірні відтінки (наприклад, надмірну жовтизну чи синяву фотографії).»

2. Модуль `image\_converter.py` (Обгортка та файлова обробка)

Цей модуль відповідає за файлове введення-виведення, конвертацію форматів, пакетну обробку та взаємодію між бібліотекою Pillow і нашими власними алгоритмами.

Функція: `get_file_size_info(filepath)`

- **Призначення:** Форматування розміру файлу у зручний для читання вигляд (**Bytes (KB)**).
- **Принцип роботи:** Викликає `os.path.getsize(filepath)` та переводить байти у кілобайти (**bytes / 1024**).

Функція: `load_image_pixels(filepath)`

- **Призначення:** Завантаження зображення та конвертація його в чистий масив кортежів (**R, G, B**).
- **Принцип роботи:**
 - Відкриває файл через `Image.open(filepath)`.
 - Якщо зображення має прозорість (**RGBA, LA, P**), створює біле полотно **RGB** і накладає зображення за допомогою `paste(..., mask)`.
 - Повертає кортеж (**pixels_list, width, height**).

Функція: `resolve_output_path(output_path, default_filename)`

- **Призначення:** Безпечне формування шляху збереження вихідного файлу.
- **Принцип роботи:** Якщо користувач вказав шлях до папки (наприклад **output/resized** або шлях зі слешем на кінці), функція автоматично дописує **default_filename** у цю папку, запобігаючи виникненню помилки **IsADirectoryError**.

Функція: `save_pixels_to_image(filepath, width, height, pixels, format_name=None)`

- **Призначення:** Збереження масиву **[(R, G, B), ...]** у файл заданого графічного формату.
- **Принцип роботи:** Створює об'єкт `Image.new('RGB', (width, height))`, заповнює його через `putdata(pixels)`, створює батьківські папки за потреби (`os.makedirs`) та записує файл на диск через `.save()`.

Функція: `convert_format(input_path, output_format, output_path=None)`

- **Призначення:** Конвертація одного зображення між форматами (JPEG, PNG, BMP, TIFF, WEBP) та розрахунок зміни розміру.
- **Принцип роботи:**
 - Зчитує зображення через `load_image_pixels`.
 - Зберігає в новому форматі через `save_pixels_to_image`.
 - Порівнює розміри через `os.path.getsize` та вираховує відсоток зміни: $\Delta\% = \{S_{\text{final}} - S_{\text{initial}}\} / S_{\text{initial}} \cdot 100\%$.

Функція: `batch_convert_formats(input_paths, output_format, output_directory='output/converted')`

- **Призначення:** Пакетна конвертація списку файлів за один виклик.
- **Принцип роботи:** У циклі викликає `convert_format` для кожного знайденого зображення та збирає результати у підсумковий список.

Функція: `resize_image(input_path, output_path, width, height, scale_percent, keep_aspect_ratio, method)`

- **Призначення:** Обгортка для масштабування окремого файлу з розрахунком пропорцій.
- **Принцип роботи:**

- Обчислює цільові розміри (**new_w, new_h**) відповідно до обраного режиму (відсоток, за шириною, за висотою або точні розміри зі збереженням або без збереження Aspect Ratio).
- Викликає власну функцію **resize_bilinear** або **resize_nearest_neighbor**.
- Зберігає результат і повертає метадані (початковий/новий розмір, об'єм).

Функція: **batch_resize_images(...)**

- **Призначення:** Пакетна зміна розміру декількох зображень за один запуск.
- **Принцип роботи:** Послідовно застосовує **resize_image** до всіх зображень у папці.

Функція: **replace_color(...), adjust_color_balance(...), auto_color_balance(...)**

- **Призначення:** Високорівневі функції, які зчитують вхідний файл, передають пікселі у відповідні функції модуля **manual_algorithms.py**, записують результат у вихідний файл та формують словник зі звітом.

3. Модуль `main.py` (Консольний інтерфейс користувача)

Модуль реалізує текстове меню для взаємодії з користувачем, валідацію введених даних та обробку помилок.

Функція: **clear_console()**

- **Призначення:** Повне очищення терміналу та буфера прокрутки.
- **Принцип роботи:** Відправляє комбінацію керуючих ANSI-послідовностей `\033c\033[2J\033[H\033[J` та викликає системну команду **clear** (`cls` на Windows).

Функція: **get_int_input(prompt, default, min_val=None, max_val=None)**

- **Призначення:** Захищене зчитування цілого числа.
- **Принцип роботи:** Якщо користувач вводить літери чи некоректні символи, функція перехоплює **ValueError**, виводить повідомлення про помилку і повторно запитує значення без аварійного закриття програми.

Функція: **get_float_input(prompt, default, min_val=None, max_val=None)**

- **Призначення:** Захищене зчитування дійсного числа з підтримкою як крапки, так і коми як десяткового роздільника (**replace(',', '.')**).

Функція: **get_rgb_input(prompt, default)**

- **Призначення:** Зчитування трьох чисел кольору **R G B** з перевіркою діапазону [0..255] для кожного каналу.

Функція: **handle_format_conversion()**

- **Призначення:** Обробник пункту меню №1. Запитує вибір поодинокого або пакетного режиму, цільовий формат (PNG/JPEG/BMP/TIFF/WEBP) та шляхи, після чого запускає конвертацію та виводить звіт зі зміною розміру в байтах та %.

Функція: **handle_resize()**

- **Призначення:** Обробник пункту меню №2. Надає вибір алгоритму (Bilinear / Nearest Neighbor), типу масштабування (% , за шириною, за висотою, точні розміри) та виконує одиночну або пакетну зміну розміру.

Функція: **handle_color_replacement()**

- **Призначення:** Обробник пункту меню №3. Отримує цільовий колір, новий колір та tolerance, виконує заміну та показує відсоток змінених пікселів.

Функція: `handle_color_balance()`

- **Призначення:** Обробник пункту меню №4. Дозволяє обрати ручне налаштування множників каналів або автоматичний розрахунок за гіпотезою «Сірого світу».

Функція: `main()`

- **Призначення:** Головний цикл програми `while True`. Відображає меню, викликає відповідні обробники та перехоплює загальні винятки (`try...except`), запобігаючи аварійному закриттю при виникненні непередбачених помилок.